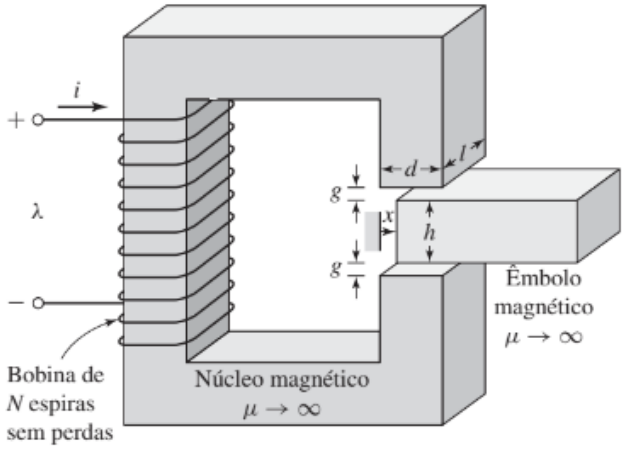
Exercício 1

Considere o circuito magnético a seguir:



N=1000 espiras;
g= 2 mm;
d= 4 cm;
L=4 cm;
Lc= 70 cm.
Massa do elemento móvel:
100 g.

Considere que:

- Que o sistema é alimentado com uma fonte de tensão constante.
- Em série com a fonte (não indicada na figura) e o enrolamento temos um resistor de 2 ohms (também não indicado na figura, representa toda a resistência do circuito, incluindo a dos enrolamentos).
- A tensão da fonte é ajustada de tal forma que se o elemento móvel está na posição x=0, teremos uma densidade de fluxo de 1.8 T no núcleo em regime permanente.
- Que existe uma mola de constante elástica 100 N/m fixada no elemento móvel.
- Quando o elemento móvel está na posição x=3 cm, a força elástica da mola é nula.
- Existe um anteparo (não indicado na figura) em x=0, de modo que o quando elemento atinge essa posição ele para, colidindo com o anteparo de forma inelástica.
- No instante inicial, t=0 a fonte de tensão é ligada, com o elemento móvel na posição x= 3cm.

Considere a seguinte característica BxH:

H [A/m]	B[T]	H [A/m]	B[T]
0	0	1100	1,689
68	0,733	1500	1,703
135	1,205	2500	1,724
203	1,424	4000	1,731
271	1,517	5000	1,738
338	1,560	9000	1,761
406	1,588	12000	1,770
474	1,617	20000	1,80
542	1,631	25000	1,816
609	1,646		

Inicialmente definimos os dados físicos do problema, a importação das bibliotecas necessárias, calculamos a área do núcleo, a tensão da fonte e por fim fizemos a interpolação dos dados BxH para a característica real do núcleo.

- Calculamos a área da seção transversal do núcleo através da fórmula $A = l.d$

Calculamos a parte a corrente que passa pela fonte na forma:

$$NI = H_f l_f + H_g l_g => I = \frac{H_f l_f + H_g l_g}{N}$$

Utilizando os dados do problema temos:

$$H_g = \frac{B}{\mu_0} = \frac{1.8}{4\pi.10^{-7}} = 1{,}432$$

$$I = \frac{(20000.0{,}7) + (1{,}432.0{,}004)}{1000}$$

$$I = 19{,}7295$$

- Através do valor da corrente obtido se obteve a tensão na fonte:

$$V = RI$$

$$V = 2.19{,}73 = 39{,}46V$$

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
from scipy.integrate import solve_ivp
μ0 = 4*np.pi*1e-7 #Permeabilidade magnética do vácuo
N, g, l, lc, d, m, k, R, V, x0 = 1000, 0.002, 0.04, 0.70, 0.04, 0.1, 100, 2, 39.46, 0.03
lg = 2*g
x_min = 0.0
# Tabela BxH
H_data = np.array([0,68,135,203,271,338,406,474,542,609,1100,1500,
                   2500,4000,5000,9000,12000,20000,25000], dtype=float)
B_data = np.array([0,0.733,1.205,1.424,1.517,1.560,1.588,1.617,1.631,1.646,
                   1.689,1.703,1.724,1.731,1.738,1.761,1.770,1.80,1.816], dtype=float)
H_interp = interp1d(B_data, H_data, kind='cubic', fill_value="extrapolate")
```

- Em seguida definimos uma função para o cálculo da área do entreferro em relação a variação da posição do êmbolo:

$$A_g = l(d - x)$$

- De mesma forma definimos uma função para o cálculo da área efetiva do núcleo em relação a posição do êmbolo. Na fórmula a seguir, notamos uma diferença pois parte do fluxo passa no núcleo e outra no entreferro
  - $A_n = l(d - \frac{x}{2})$

```
def Ag(x):
    x_safe = np.maximum(1e-6, np.minimum(x, d*0.999))
    return l * (d - x_safe)

def An(x):
    x_safe = np.maximum(x_min, x)
    return l*(d - 0.5*x_safe)
```

Além disso, criamos uma função que calcula a coenergia magnética para ser usadas em momentos futuros do código. Com isso, utilizamos a fórmula:

$$W' = \int i d\lambda => W' = \int iNAdB$$

- Assim, criamos uma lista de 0 até 1.8 em 100 pontos para que pudesse ser usado no cálculo da integral
- Definimos uma relação do campo magnético para cada ponto
- Calculou-se o valor da corrente em cada ponto
- E por fim, utilizamos a função *np.trapezoid* para o cálculo de cada área que representa a coenergia.

```python
# Essa função calcula a "energia guardada" no ímã (Coenergia)
def coenergia_total(B, x):
    # Cria uma listinha de 0 até o valor atual do Fluxo (B) com 100 pedacinhos
    B_integr = np.linspace(0, B, 100)

    # Olha na tabela BxH para ver qual o esforço (H) o ferro faz para cada pedaço de B
    H_vals = H_interp(B_integr)

    # Faz a conta da corrente (i) somando o que se perde no ferro e o que se perde no ar
    # i = (Perda no Ferro + Perda no Ar) / Número de voltas da bobina
    i_vals = (H_vals*lc + (B_integr*An(x)*lg)/(μ0*Ag(x))) / N

    # Soma todas as áreas desses 100 pedacinhos para achar a energia total
    # O trapézio serve para calcular a área embaixo da curva do gráfico Corrente vs Fluxo
    return np.trapezoid(i_vals * N * An(x), B_integr)
```

Apresentação dos gráficos da posição do elemento móvel, densidade de fluxo no núcleo, corrente na bobina e da força magnética em função do tempo: considerando a característica ideal do núcleo.

Nesse setor por estarmos considerando o núcleo como ideal, é determinado que $\mu \rightarrow \infty$.

- Em seguida, a princípio definimos que se o êmbolo atingir o anteparo (x=0) a velocidade fica nula, ou seja, simula a colisão;

Determinamos o valor do indutor em relação a área:

$$L = \frac{\mu_0 N^2 A}{l_g} \qquad com \qquad A = l(d - x)$$

- Usamos a função *np.clip* apenas para impedir valores inválidos de x

Com isso, derivamos a indutância em relação a posição:

$$\frac{dL}{dx} = -\frac{\mu_0 N^2 l}{l_g}$$

Utilizando a lei de Faraday para achar a derivação da corrente em função do tempo:

$$V = Ri + \frac{d\lambda}{dt} \rightarrow \lambda = Li$$

$$\frac{d\lambda}{dt} = L\frac{di}{dt} + i\frac{dL}{dx}\frac{dx}{dt}$$

$$\frac{di}{dt} = \frac{V - Ri - i\frac{dL}{dx}v}{L}$$

- A equação é resolvida pela função *solve_ivp* e com isso é obtido o gráfico da corrente em função do tempo

Em seguida calculamos a derivada da velocidade em relação ao tempo através da lei de Newton:

$$m\frac{dv}{dt} = F_{\text{mag}} - F_{\text{mola}}$$

$$\frac{dv}{dt} = \frac{F_m - k(x - x_0)}{m}$$

- Com isso, a função *solve_ivp* integra duas vezes a equação obtendo assim a relação da posição em função do tempo

Além disso, calculamos a densidade de fluxo:

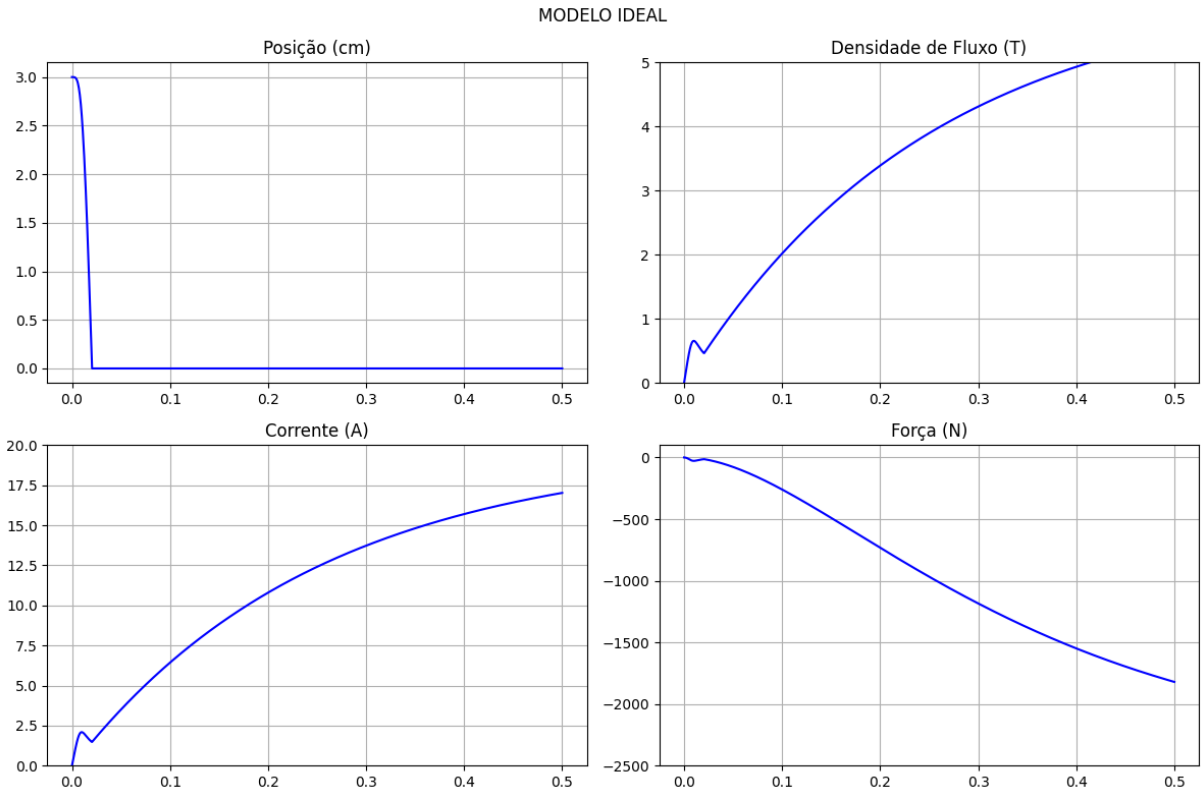
$$B(t) = \frac{\mu_0 N i(t)}{l_g}$$

- No código foi utilizado a função *sol1.y[2] = i(t)*

Através da derivada da indutância calculada anteriormente podemos calcular a força magnética para cada intervalo de tempo:

$$F_m = 0.5\, i^2\, \frac{dL}{dx} \leftrightarrow\ F_m = 0.5\,(\text{sol1.y}[2])^2 \left(-\frac{\mu_0 N^2 l}{l_g}\right)$$

```python
def sistema1(t, X):
    x, v, i = X
    if x <= 0: x, v = 0, 0
    #A indutância muda com a posição, mas ignora o ferro
    L = (μ0 * N**2 * l * (d - np.clip(x, 0, d*0.999))) / lg
    dLdx = - μ0 * N**2 * l / lg
    #Força calculada puramente pela variação da indutância
    Fm = 0.5 * i**2 * dLdx
    #Baseado na Lei de Faraday
    didt = (V - R*i - i*dLdx*v) / L
    dvdt = (Fm - k*(x - x0)) / m
    if x <= 0 and dvdt < 0: dvdt = 0
    return [v, dvdt, didt]

t_final = 0.5
t_eval = np.linspace(0, t_final, 1000)

sol1 = solve_ivp(sistema1, (0, t_final), [x0, 0, 0], t_eval=t_eval, method="Radau")
B1 = μ0 * N * sol1.y[2] / lg
Fm1 = 0.5 * sol1.y[2]**2 * (-μ0 * N**2 * l / lg)

plot_simples(sol1.t, sol1.y[0], B1, sol1.y[2], Fm1, "MODELO IDEAL", "blue")
```



Apresentação dos gráficos da posição do elemento móvel, densidade de fluxo no núcleo, corrente na bobina e da força magnética em função do tempo: considerando a característica linear do núcleo.

No modelo ideal temos que o ferro não é mais ideal, porém não utiliza também a curva B x H por completo, mas sim usa uma aproximação através da permeabilidade constante.

- Escolhemos dois pontos da curva B x H para aproximar o ferro através de uma reta
- Calculamos a inclinação da reta obtendo a permeabilidade média do ferro:

$$\mu_{eq} = \frac{\Delta B}{\Delta H}$$

Em seguida calculamos a relutância do núcleo e do ar:

$$R_n = \frac{l_c}{\mu_{eq} A} \rightarrow A = l\, d$$

$$R_g = \frac{l_g}{\mu_0 A_g} \rightarrow A_g = l(d - x)$$

Com isso, obtivemos a expressão para a indutância:

$$L = \frac{N^2}{R_{\text{tot}}}\ com\ R_{\text{tot}} = R_g + R_n$$

- A indutância está em função da posição devido a relutância do ar;
- Assim, conseguimos obter a derivada da relutância em função da posição que será usada para a força magnética;

De forma análoga ao núcleo ideal, obtemos a derivada da corrente em função do tempo através da Lei de Faraday e através da Lei de Newton a derivada da velocidade em função do tempo:

$$\frac{di}{dt} = \frac{V - Ri - i\frac{dL}{dx}v}{L}$$

$$\frac{dv}{dt} = \frac{F_m - k(x - x_0)}{m}$$

- As equações são resolvidas pela função *solve_ivp* e com isso é obtido o gráfico da corrente e posição, sequencialmente, em função do tempo;

Além disso, calculamos a densidade de fluxo magnético levando em conta a relutância do núcleo:

$$B = \frac{Ni}{(R_n + R_g)A}$$

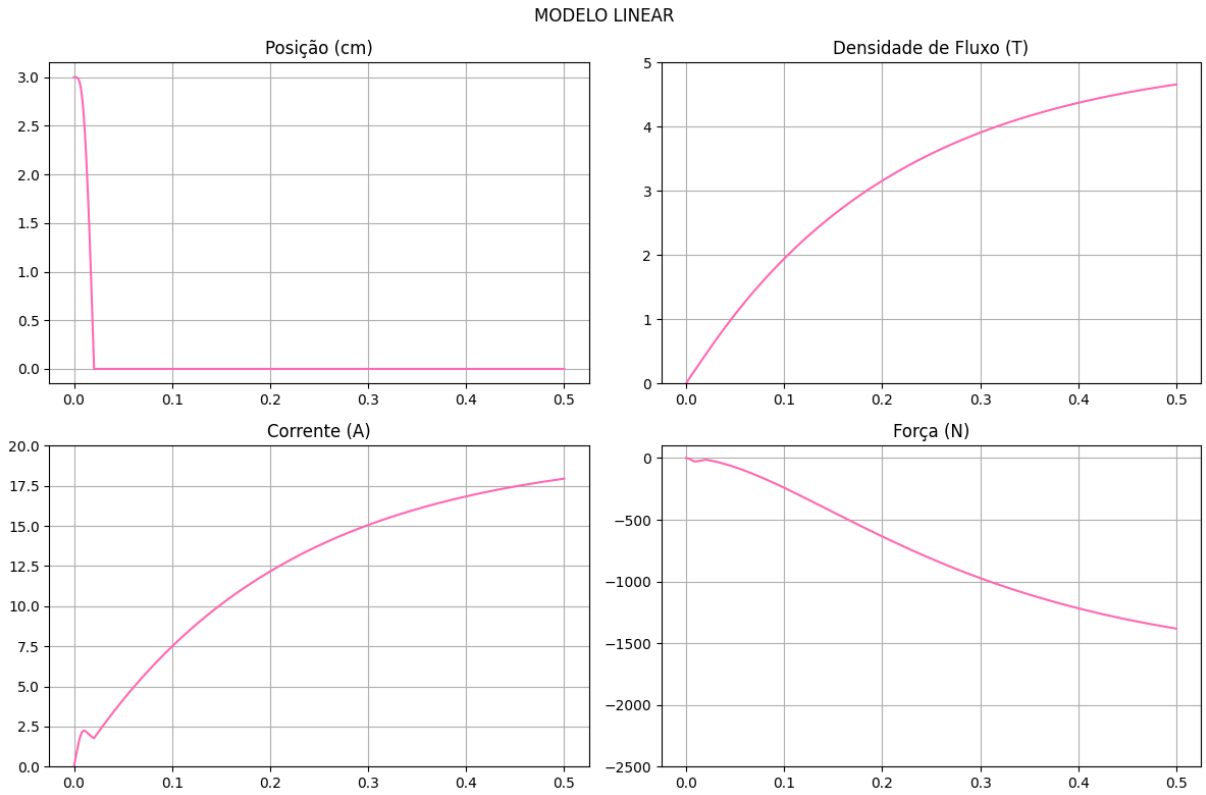
E por fim calculamos a força magnética através da derivada da indutância já obtida anteriormente:

$$F_m = 0.5\, i^2 \frac{dL}{dx}$$

- No código foi criado um vetor F_{m2} onde está sendo armazenado o valor da força para cada ponto no tempo;
- O laço no código foi criado para que pudesse ser usado cada corrente e posição em relação a ponto a ponto, obtendo também a derivada da indutância para cada ponto;

```
H1_lin, Bp1_lin, H2_lin, Bp2_lin = 135, 1.205, 542, 1.631
μeq = (Bp2_lin-Bp1_lin)/(H2_lin-H1_lin)#Calcula uma permeabilidade média constante para o ferro
Rn = lc / (μeq * l * d)

def sistema2(t, X):
    x, v, i = X
    if x <= 0: x, v = 0, 0
    Rg = lg / (μ0 * l * (d - np.clip(x, 0, d*0.999)))
    L = (N**2) / (Rn + Rg)
    dLdx = -N**2 * lg * μ0 * l / (((Rn * μ0 * l * (d - np.clip(x, 0, d*0.999))) + lg)**2)
    Fm = 0.5 * i**2 * dLdx
    didt = (V - R*i - i*dLdx*v) / L
    dvdt = (Fm - k*(x - x0)) / m
    if x <= 0 and dvdt < 0: dvdt = 0
    return [v, dvdt, didt]
```

```
sol2 = solve_ivp(sistema2, (0, t_final), [x0, 0, 0], t_eval=t_eval, method="Radau")
B2 = N * sol2.y[2] / ((Rn + (lg / (μ0 * l * (d - np.clip(sol2.y[0], 0, d*0.999))))) * l * d)
# Força calculada com a derivada de L para o linear
Fm2 = []
for xi, ii in zip(sol2.y[0], sol2.y[2]):
    dL = -N**2 * lg * μ0 * l / (((Rn * μ0 * l * (d - np.clip(xi, 0, d*0.999))) + lg)**2)
    Fm2.append(0.5 * ii**2 * dL)

plot_simples(sol2.t, sol2.y[0], B2, sol2.y[2], Fm2, "MODELO LINEAR", "hotpink")
```



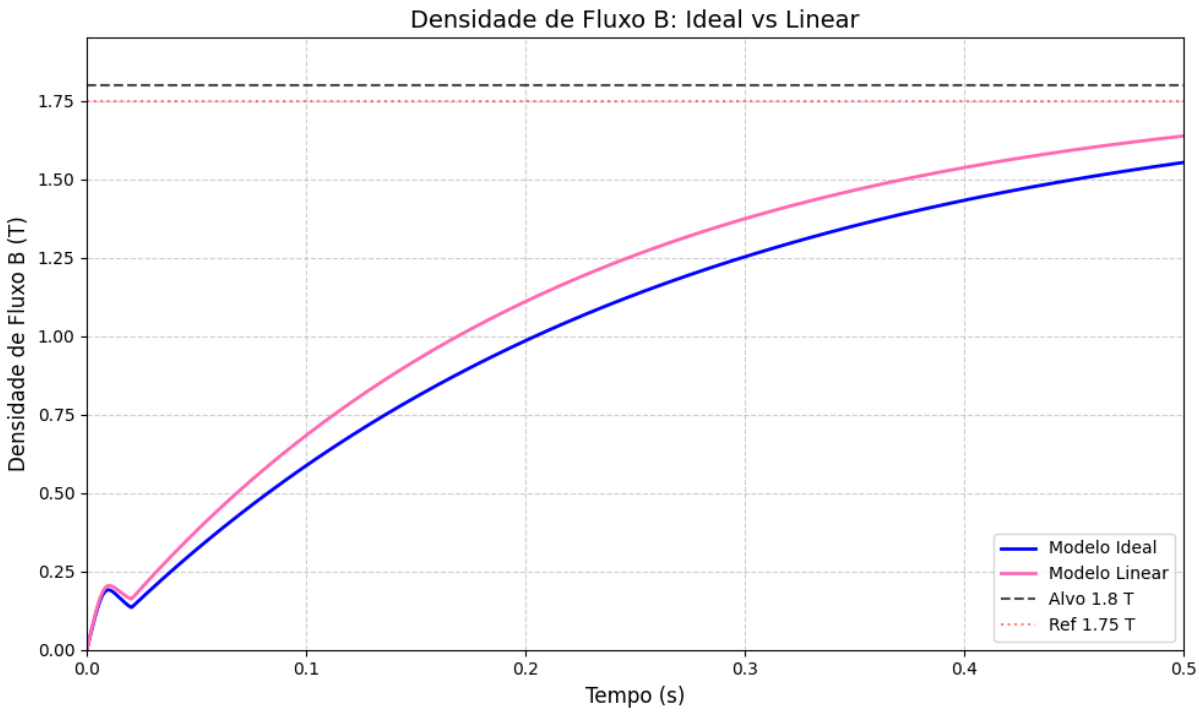
Correção dos gráficos do fluxo ideal e real para atingirem 1.8T

```python
# --- 1. PARÂMETROS DE AJUSTE (PARA IDEAL E LINEAR) ---
B_alvo = 1.8
I_regime = V / R
Area_total = l * d
Rg_impacto = lg / (μ0 * Area_total)

# R_sat_ajuste calibra os modelos simplificados para o alvo de 1.8T no gráfico
#Eles não "sabem" o que é saturação.
#Se não usássemos a rsat, o fluxo no modelo Ideal iria para um valor infinito ou irreal, pois ele ignora a oposição do ferro.
#Portanto, para respeitar a premissa do enunciado de que o sistema em regime permanente opera em 1.8 T
# eu apliquei essa relutância de ajuste. Isso garante que todos os modelos partam da mesma condição física final
#permitindo comparar de forma justa como cada um deles chega até lá.
R_sat_ajuste = (N * I_regime) / (B_alvo * Area_total) - Rg_impacto

# --- 2. CÁLCULO DOS VETORES DE FLUXO PARA O PLOT ---
# Fluxo B do Modelo Ideal (Normalizado)
B1_plot = (N * sol1.y[2]) / ((Rg_impacto + R_sat_ajuste) * Area_total)

# Fluxo B do Modelo Linear (Normalizado)
B2_plot = (N * sol2.y[2]) / ((Rg_impacto + R_sat_ajuste) * Area_total)
```

```python
# --- 3. EXECUÇÃO DO PLOT COM ZOOM EQUILIBRADO ---
plt.figure(figsize=(10, 6))

# Plotagem das curvas
plt.plot(sol1.t, B1_plot, 'b', linewidth=2, label="Modelo Ideal")
plt.plot(sol2.t, B2_plot, 'hotpink', linewidth=2, label="Modelo Linear")

# Linhas de Referência
plt.axhline(1.8, color='black', linestyle='--', alpha=0.7, label='Alvo 1.8 T')
plt.axhline(1.75, color='red', linestyle=':', alpha=0.5, label='Ref 1.75 T')

# Configurações de Eixos e Título
plt.title("Densidade de Fluxo B: Ideal vs Linear", fontsize=14)
plt.xlabel("Tempo (s)", fontsize=12)
plt.ylabel("Densidade de Fluxo B (T)", fontsize=12)

# ZOOM INVERSO (Escala completa mas ajustada ao teto)
plt.ylim([0, 1.95])  # Vai de 0 até um pouco acima do alvo (1.8), sem sobrar muito espaço
plt.xlim([0, 0.5])   # Ajuste o 0.5 conforme o tempo que seu sistema leva para estabilizar
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(loc='lower right')

plt.tight_layout()
```

A correção foi feita através da criação de uma relutância de saturação que permitisse a densidade de fluxo não ultrapassar os 1.8T:

$$B = \frac{Ni}{(R_g + R_{\text{sat ajuste}})A}$$



Apresentação dos gráficos da posição do elemento móvel, densidade de fluxo no núcleo, corrente na bobina e da força magnética em função do tempo: considerando a característica real do núcleo.

No modelo real é utilizado a característica da curva BxH para obtenção dos parâmetros.

A princípio calculamos a queda da força magnetomotriz no núcleo e no entreferro:

$$F_{\text{núcleo}} = H_n \, l_c$$

$$F_{\text{gap}} = H_g l_g = \frac{B_n A_n l_g}{\mu_0 A_g}$$

Com isso, é possível obter a corrente do sistema:

$$Ni = F_{\text{núcleo}} + F_{\text{gap}}$$

$$i = \frac{H_n l_c}{N} + \frac{B_n A_n l_g}{\mu_0 A_g N}$$

Já para o cálculo da força magnética, utilizamos a derivada da coenergia em dois pequenos instantes diferentes. Com isso, a derivada é obtida através da diferença central em os dois pontos:

$$F_m = -\frac{\partial W'}{\partial x}$$

$$F_m \approx -\frac{W'(x+dx) - W'(x-dx)}{2dx}$$

- No código utilizamos um valor extremamente pequeno para dx para que pudéssemos avaliar o quanto a energia mudaria com uma pequena alteração na posição

Além disso, pudemos calcular a força da mola através da Lei de Hooke:

$$F_{el} = -k(x - x_0)$$

- O sinal negativo na expressão na expressão aparece porque a mola tenta trazer o êmbolo de volta para a posição de equilíbrio x_0

Por conseguinte, utilizamos a equação elétrica do atuador magnético (advindo da Lei de Faraday) para obter a derivada da densidade de fluxo magnético:

$$V = Ri + \frac{d\lambda}{dt}$$

$$\lambda = N\Phi = NB_nA_n(x)$$

$$\frac{d\lambda}{dt} \approx NA_n(x)\frac{dB_n}{dt}$$

$$V - Ri = NA_n(x)\frac{dB_n}{dt}$$

$$\frac{dB_n}{dt} = \frac{V - Ri}{NA_n(x)}$$

- No código essa expressão é resolvida através da função *solve_ivp* , obtendo o gráfico de B(t)

Além disso, através da Lei de Newton foi possível obter a derivada da velocidade em função do tempo:

$$m\frac{dv}{dt} = F_m - F_{el}$$

$$\frac{dv}{dt} = \frac{F_m - F_{el}}{m}$$

- Através da expressão (integrando duas vezes), obtemos o gráfico x(t)

```python
def system(t, y):

    # Pega o que temos agora: Fluxo, Onde ele está e a Velocidade
    Bn = y[0]
    x  = y[1]
    v  = y[2]

    # 1. Descobre qual a corrente que tá passando no fio agora
    Hn = float(H_interp(Bn))

    parte_nucleo = Hn * lc
    parte_gap = (Bn * An(x) * lg) / (μ0 * Ag(x))

    i = (parte_nucleo + parte_gap) / N

    # 2. Calcula o "puxão" do ímã (Força Magnética)
    # Ele faz um teste: "Se eu mexer só um tiquinho, a energia muda quanto?"
    dx_s = 1e-5

    W_p = coenergia_total(Bn, x + dx_s)
    W_m = coenergia_total(Bn, x - dx_s)



     # Esse resultado é a força do puxão
     Fm = -(W_p - W_m) / (2 * dx_s)

     # 3. Calcula o empurrão da mola (ela quer levar o bloco de volta)
     Fel = -k * (x - x0)

     # 4. Avisa pro programa como as coisas devem mudar:

     # Como o fluxo aumenta ou diminui
     dBn_dt = (V - R*i) / (N * An(x))

     # A velocidade diz pra onde ele anda
     dx_dt = v

     # A briga entre o ímã e a mola faz ele acelerar
     dv_dt = (Fm - Fel) / m

     return [dBn_dt, dx_dt, dv_dt]
```

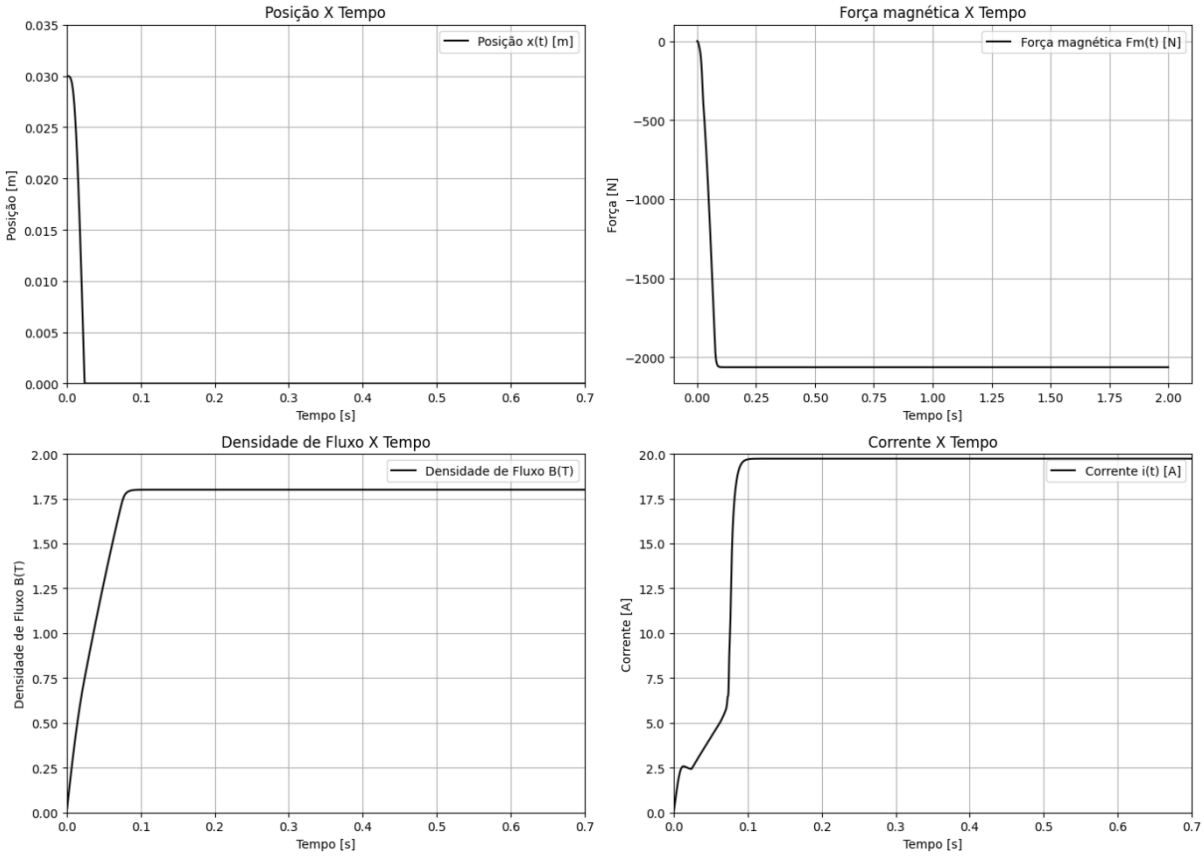
```python
# Começa parado e sem nada de fluxo
y0 = [0.0, x0, 0.0]

# Vai fingir que passou 2 segundos
T_final = 2

# Cria 6 mil pontinhos no tempo pra desenhar o gráfico
t_eval = np.linspace(0, T_final, 6000)


sol = solve_ivp(system,
                [0, T_final],
                y0,
                method='Radau',
                t_eval=t_eval,
                rtol=1e-8,
                atol=1e-10)

# Separa os resultados pra usar
Bn = sol.y[0]
x  = sol.y[1]
v  = sol.y[2]
```

No bloco anterior do código nós obtivemos os resultados referentes a B(t), x(t) e v(t), nessa nova parte o código recalcula a cada instante de tempo a força no entreferro, a força associada ao núcleo e com isso obtendo a força magnética e a corrente. Nesse bloco, foi utilizado as mesmas expressões anteriores criando listas para recalcular em cada instante de tempo

```python
# O programa lá em cima só guardou Fluxo, Onde ele está e Velocidade.
# Agora a gente tem que calcular na mão a Força e a Corrente de cada momento.

# Cria lista varias pra guardar forças e corrente
F_gap     = np.zeros_like(Bn)
F_nucleo  = np.zeros_like(Bn)
Fm_total  = np.zeros_like(Bn)
i         = np.zeros_like(Bn)

dx_small = 1e-5
# Agora que eu sei Bn e x em cada instante,vou calcular as forças e corrente em cada instante
# Vamos olhar ponto por ponto e fazer a conta de novo:
for k in range(len(Bn)):

    # Força que o ar faz
    F_gap[k] = (Bn[k]**2 * Ag(x[k])) / (2 * μ0)

    # Força que o ferro faz (usando aquela conta da energia)
    W_plus  = coenergia_total(Bn[k], x[k] + dx_small)
    W_minus = coenergia_total(Bn[k], x[k] - dx_small)

    F_nucleo[k] = (W_plus - W_minus) / (2 * dx_small)
```

```
    # Soma as duas pra saber o puxão total
    Fm_total[k] = -F_gap[k] - F_nucleo[k]

    # Descobre qual era a corrente nesse momento
    Hn = float(H_interp(Bn[k]))

    parte_nucleo = Hn * lc
    parte_gap = (Bn[k] * An(x[k]) * lg) / (μ0 * Ag(x[k]))

    i[k] = (parte_nucleo + parte_gap) / N

plt.figure(figsize=(14, 10))
tempo_zoom = 0.7

plt.subplot(2, 2, 1)
plt.plot(sol.t, x, color='black', label='Posição x(t) [m]')
plt.title('Posição X Tempo')
plt.xlabel('Tempo [s]')
plt.ylabel('Posição [m]')
plt.xlim([0, tempo_zoom])
plt.ylim([0, 0.035])
plt.grid(True)
plt.legend()


plt.subplot(2, 2, 2)
plt.plot(sol.t, Fm_total, color='black', label='Força magnética Fm(t) [N]')
plt.title('Força magnética X Tempo')
plt.xlabel('Tempo [s]')
plt.ylabel('Força [N]')
plt.grid(True)
plt.legend()
```

```python
plt.plot(sol.t, Bn, color='black', label='Densidade de Fluxo B(T)')
plt.title("Densidade de Fluxo X Tempo")
plt.xlabel('Tempo [s]')
plt.ylabel('Densidade de Fluxo B(T)')
plt.xlim([0, tempo_zoom])
plt.ylim([0, 2.0])
plt.grid(True)
plt.legend()


plt.subplot(2, 2, 4)
plt.plot(sol.t, i, color='black', label='Corrente i(t) [A]')
plt.title('Corrente X Tempo')
plt.xlabel('Tempo [s]')
plt.ylabel('Corrente [A]')
plt.xlim([0, tempo_zoom])
plt.ylim([0, 20])
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```



Durante a simulação é necessário impor uma restrição física que represente o batente mecânico do sistema, impedindo que o elemento móvel ultrapasse uma posição mínima. Para isso, o código verifica se algum valor da posição calculada é menor que esse limite e, caso isso ocorra, corrige a posição para o valor do batente:

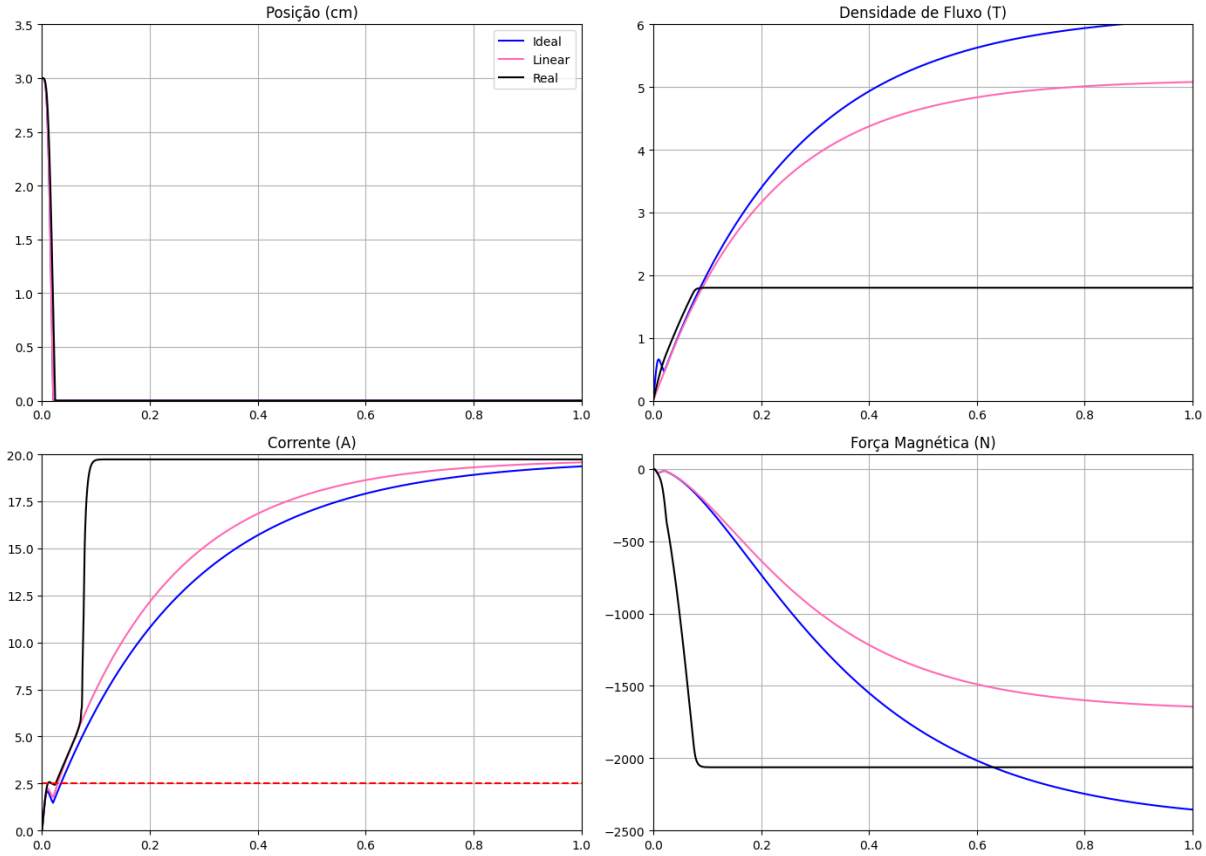
$$x \geq x_{\{min\}}$$

Além disso, considera-se que a colisão com o batente é inelástica, fazendo com que a velocidade seja anulada no momento do impacto. Dessa forma, o modelo respeita as limitações físicas do sistema, impedindo que o elemento móvel atravesse o anteparo mecânico.

```
# O batente (Pra não deixar um ferro atravessar o outro)

# Se a conta deu que ele passou do zero, vira zero, bateu
x[x < x_min] = x_min

# Se bateu, a velocidade para na hora
v[x < x_min] = 0
```

Também plotamos as três formas do núcleo (ideal, real, linear) juntas para que fosse possível observar e comparar as três formas.

```python
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

# AJUSTE O ZOOM AQUI
tempo_zoom = 1.0

# 1. POSIÇÃO (cm)
axs[0,0].plot(sol1.t, sol1.y[0]*100, 'blue', label='Ideal')
axs[0,0].plot(sol2.t, sol2.y[0]*100, 'hotpink', label='Linear')
axs[0,0].plot(sol.t, x*100, 'black', label='Real')
axs[0,0].set_title("Posição (cm)")
axs[0,0].set_xlim([0, tempo_zoom]) # ZOOM NO TEMPO
axs[0,0].set_ylim([0, 3.5])        # ZOOM NA ALTURA (3.5cm)
axs[0,0].legend()
axs[0,0].grid(True)

# 2. FLUXO (T)
axs[0,1].plot(sol1.t, B1, 'blue')
axs[0,1].plot(sol2.t, B2, 'hotpink')
axs[0,1].plot(sol.t, Bn, 'black')
axs[0,1].set_title("Densidade de Fluxo (T)")
axs[0,1].set_xlim([0, tempo_zoom]) # ZOOM NO TEMPO
axs[0,1].set_ylim([0, 6.0])        # ZOOM NA ALTURA (Saturação em 1.8)
axs[0,1].grid(True)


# 3. CORRENTE (A)
axs[1,0].plot(sol1.t, sol1.y[2], 'blue')
axs[1,0].plot(sol2.t, sol2.y[2], 'hotpink')
axs[1,0].plot(sol.t, i, 'black')
axs[1,0].axhline(2.5, color='red', linestyle='--', label='Ref 2.5A')
axs[1,0].set_title("Corrente (A)")
axs[1,0].set_xlim([0, tempo_zoom]) # ZOOM NO TEMPO
axs[1,0].set_ylim([0, 20])         # ZOOM NA ALTURA
axs[1,0].grid(True)

# 4. FORÇA (N) - Aqui a força do Real ('black') aparecerá negativa
axs[1,1].plot(sol1.t, Fm1, 'blue')
axs[1,1].plot(sol2.t, Fm2, 'hotpink')
axs[1,1].plot(sol.t, Fm_total, 'black')
axs[1,1].set_title("Força Magnética (N)")
axs[1,1].set_xlim([0, tempo_zoom]) # ZOOM NO TEMPO
axs[1,1].set_ylim([-2500, 100])
axs[1,1].grid(True)

plt.tight_layout()
plt.show()
```



Considerando a característica real do núcleo, determine a energia dissipada no momento da colisão com o anteparo e em seguida para uma mola de 1000 N/m.

A princípio identificou o instante que o êmbolo chegou em x=0

- Esse instante no código foi chamado de *idx*

Por conseguinte, calculou-se a energia cinética:

$$E_c = \frac{1}{2}mv^2$$

- No código foi utilizado *idx – 1* para que pudesse pegar o instante em que a velocidade ainda não fosse nula, ou seja, foi calculado a energia cinética imediatamente antes da batida

Além disso, foi obtido a energia no campo magnético no momento do impacto através da coenergia:

$$W' = \int i d\lambda$$

Em seguida, calculou-se a energia dissipada no resistor:

$$i = \frac{H l_c}{N} + \frac{B_n \, \mu_0 \, A_g}{A_n \, l_g}$$

$$P_R = R i^2$$

$$E_R = \int R \, i^2 \, dt$$

- Foi utilizado a função *np.trapezoid* para calcular a área formada pela integral e o obter o valor da energia dissipada no resistor;

Por fim, a energia total dissipada:

$$E_{\text{total}} = E_c + E_m + E_R$$

Para a mola de 1000 N/m foi feito apenas a mudança da constante de elasticidade usando o mesmo processo.

```
def calcular_balanco_energia(sol_data, k_valor, nome):
    # 1. Extração dos dados da simulação
    # sol_data.y[0] = Bn (Fluxo), y[1] = x (Posição), y[2] = v (Velocidade)
    Bn_local = sol_data.y[0]
    x_local = sol_data.y[1]
    v_local = sol_data.y[2]
    t_local = sol_data.t

    # 2. Identificar o índice do impacto (primeira vez que x <= 0)
    idx = np.where(x_local <= 1e-6)[0][0]

    # --- CÁLCULO DAS ENERGIAS NO INSTANTE DO IMPACTO ---

    # A. Energia Cinética (Ec = 1/2 * m * v^2)
    # Pegamos o índice [idx-1] porque no [idx] a velocidade já foi zerada no código
    v_antes = v_local[idx-1]
    E_cinetica = 0.5 * m * (v_antes**2)

    # B. Energia Magnética (Coenergia armazenada no campo)
    E_magnetica = coenergia_total(Bn_local[idx], x_local[idx])

    # C. Energia Dissipada no Resistor (Integral de R * i^2 dt)
    # Calculamos a corrente i para todo o percurso até o impacto
```

```python
    H_inst = H_interp(Bn_local[:idx+1])
    i_inst = (H_inst*lc + (Bn_local[:idx+1]*An(x_local[:idx+1])*lg)/(μ0*Ag(x_local[:idx+1]))) / N
    E_resistor = np.trapezoid(R * (i_inst**2), t_local[:idx+1])

    # D. Energia Total
    E_total = E_cinetica + E_magnetica + E_resistor

    print(f"--- {nome} (k = {k_valor} N/m) ---")
    print(f"Energia Cinética:  {E_cinetica:.6f} J")
    print(f"Energia Magnética: {E_magnetica:.6f} J")
    print(f"Energia Resistor:  {E_resistor:.6f} J")
    print(f"ENERGIA TOTAL:     {E_total:.6f} J\n")
    return E_total

# --- EXECUÇÃO ---

# Questão 1: Modelo Real (k = 100)
E_diss_1 = calcular_balanco_energia(sol, 100, "RESULTADO IMPACTO REAL")

# Questão 2: Modelo com mola 1 kN/m
k_original = 100
globals()['k'] = 1000 # Altera para 1000 para a nova simulação
sol_k1000 = solve_ivp(system, [0, T_final], y0, method='Radau', t_eval=t_eval)
```

```python
E_diss_2 = calcular_balanco_energia(sol_k1000, 1000, "RESULTADO IMPACTO 1kN/m")
globals()['k'] = k_original # Restaura o original
```

```
--- RESULTADO IMPACTO REAL (k = 100 N/m) ---
Energia Cinética:  0.317911 J
Energia Magnética: 1.469882 J
Energia Resistor:  0.231057 J
ENERGIA TOTAL:     2.018851 J

--- RESULTADO IMPACTO 1kN/m (k = 1000 N/m) ---
Energia Cinética:  0.667578 J
Energia Magnética: 1.253944 J
Energia Resistor:  0.192696 J
ENERGIA TOTAL:     2.114217 J
```